

Abstract: Stream Processing Pipeline for Municipal Environmental Sensors

Markus Wohlgenannt

Oct 4, 2026

GitHub repository:

https://github.com/Morwoi/Projekt_Data_Engineering_DLBDSEDE02

Objective and scenario

The project builds the data side of a municipal environmental sensor network. This pipeline continuously ingests measurements, stores them reliably and makes them usable for planning dashboards and a citizen warning app. To give it a concrete operational purpose, it is placed in Hamburg-Wilhelmsburg. This Elbe island combines dense housing, a high-rise estate, port industry, a park and dikes within a few kilometres. Five stations stand for these five kinds of places and serve specific users. A climate adaptation planner monitors heat days and cloudbursts. A port terminal shift supervisor tracks wind gusts. A care service for elderly residents needs especially reliable heat warnings, and residents in general need timely alerts. The planner's figures feed a district assembly whose factions weigh trees, drainage and port interests differently. The system succeeds if every reading is stored exactly once and each user can tell how far a figure can be trusted. It fails if the stream stalls on a temporary fault, if data is lost, or if a number looks more certain than it is.

Technical approach

Since the municipality's sensors are not available, the Open-Meteo API stands in for them. A Python producer polls it every ten seconds per station and publishes each reading as JSON to a Kafka topic partitioned by station. Kafka runs as a single KRaft node locally. Upgrading to a multi-broker cluster is a configuration change rather than a code change. A Python consumer writes each message to MongoDB. The schema-less documents can take new sensor types such as CO2 or noise without migrations. Offsets are committed only after a successful write to guarantee at-least-once delivery. Writes are upserts on station and timestamp so redelivery cannot create duplicates. Failed writes are retried and then parked in a dead-letter collection. If MongoDB stays down even after all retries, the consumer does not commit and rewinds, allowing Kafka to buffer the data until the database returns. If the API is unreachable, the producer publishes an explicit offline record without values instead of inventing any. A simulator mode remains only for offline demonstrations. Everything starts with a simple `docker compose up` command.

Integration into the end-user landscape

Host-side Python scripts act as the backend a user interface would call. The script `planner_queries.py` aggregates daily statistics per station from real readings only. It reports how much of each day rests on real measurements and tracks how fast conditions change. This includes the heaviest rainfall and the largest temperature swing within one hour, plus the DWD flags for hot days, tropical nights and heavy rain. Plan-

ners query these rollups through MongoDB aggregation pipelines; because trend and regression analyses need seasons to years of history, raw readings are kept for 90 days and the daily rollups indefinitely. The script `materialize_daily_stats.py` persists these rollups beyond the raw-data retention. For the citizens, `citizen_status.py` classifies each station as ok, stale or unavailable based on the age of the last real reading, applying a stricter profile for vulnerable people.

The repository contains the real weather data of July to September 2026 for the five stations, complete with documented example ranges. On 30 July a cold front dropped the temperature in Kirchdorf-Süd by 8.5 °C within one hour. The daily mean usually hides such sudden events. On 15 September almost all of the Inselpark's 12.1 mm fell within a single hour. The differences between stations are small because a 2 km model smooths out the urban heat island. Real sensors would show more localized variations.

Result

The pipeline processes a continuous stream from several stations and does not break when the data source or the database is temporarily unavailable. A test script forces a real API outage against the running stack and confirms that the citizen status switches to unavailable and back. Stopping MongoDB for 75 seconds, longer than all write retries, lost no reading. Every ten-second cycle of the outage was stored successfully once the database returned.

Reflection

The most instructive problems produced no error at all. The `kafka-python` client hung silently. Switching to `confluent-kafka` solved this issue. The consumer initially received nothing because Kafka's default replication factor of 3 for internal topics cannot be met by a single broker. Setting this value to 1 fixed the problem. A local MongoDB installation occupied port 27017, meaning a host script could silently have read the wrong database. The stack now uses port 27018 to prevent this. Furthermore, Open-Meteo only publishes a new observation every 15 minutes. This caused the daily rainfall sum to count each value about 90 times. The aggregation now ensures each observation is counted exactly once. The first hourly rainfall figures exceeded some daily totals because the window reached across midnight; the windows now stay within their day. In every case the pipeline seemingly ran without issues. Only comparing stored values with expectations revealed the faults. The same applied to my claim that no data is lost. Only stopping the database for longer than the retries showed that a message could be committed without even a dead-letter entry; the consumer now rewinds instead of committing.

The tutor's feedback shaped both later phases. After phase 1 the concept was too close to the data and too far from its real use. I therefore implemented the users' queries and reliability rules as scripts instead of just adding descriptions. After phase 2 I placed the system in a real district with concrete users and added sample data with example ranges. I also took up the two critical remarks in the code. Inventing values for a broken sensor was wrong, so the default fallback is now an explicit offline record. The tutor was also right that, for a planner, rapid changes matter more than daily averages. Therefore, the daily rows now carry hourly intensity metrics.

I learned that a data system has to be judged by what its users will do with the numbers, not just by whether it runs. For similar projects I will define the user and the decision first, test end to end with known values, and state limitations clearly where the reader will see them.